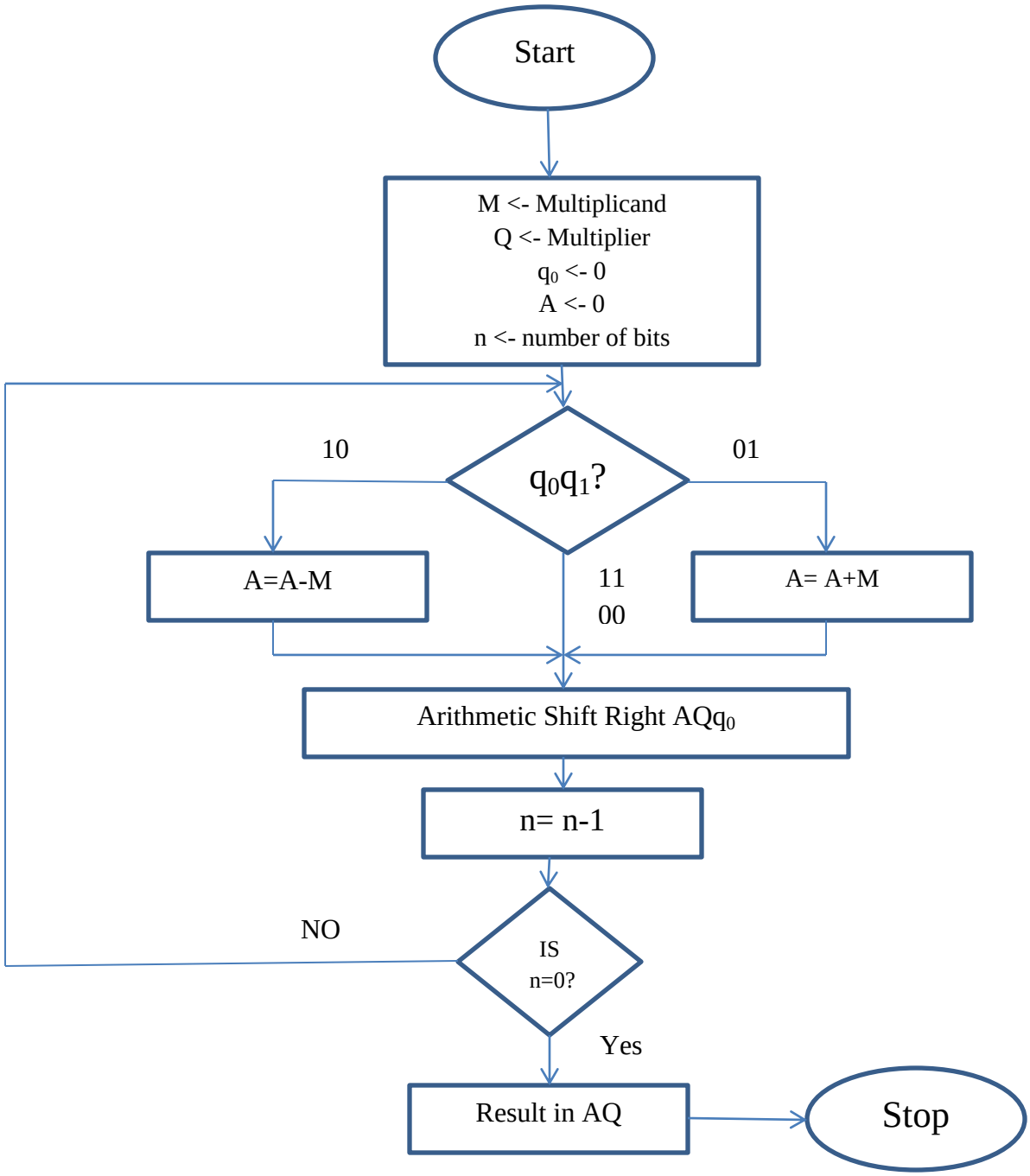
Booth's Algorithm for Signed Multiplication

The booth algorithm is a multiplication algorithm that allows us to multiply the two signed binary integers in 2's complement, respectively. It is also used to speed up the performance of the multiplication process. It is very efficient too. It works on the string bits 0's in the multiplier that requires no additional bit only shift the right-most string bits and a string of 1's in a multiplier bit weight 2^k to weight 2^m that can be considered as $\mathbf{2^{k+1} - 2^m}$.

Flowchart:



Example: (-7) x (+3) = (-21)

Here, Multiplicand = -7, Multiplier= +3 and product= -21

M= -7_{10}

= 2's complement of 0111_2

= 1001_2

-M = 0111_2

Tracing Table:

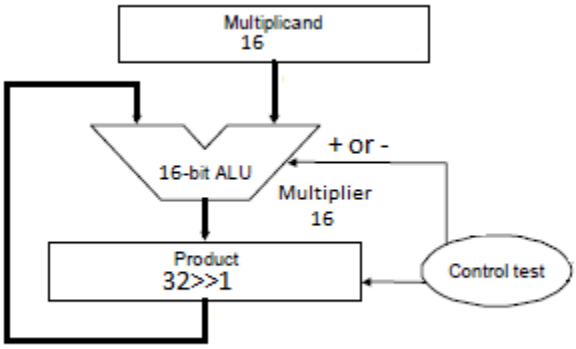
n	A	Q	q_0	Action
4	0000	0011	0	Initialization
	0111	0011	0	A=A-M
3	0011	1001	1	ASR AQq_0
2	0001	1100	1	ASR AQq_0
	1010	1100	1	A=A+M
1	1101	0110	0	ASR AQq_0
0	1110	1011	0	ASR AQq_0

Result: 1110 1011

0001 0101 (2's Complement)

= -21_{10}

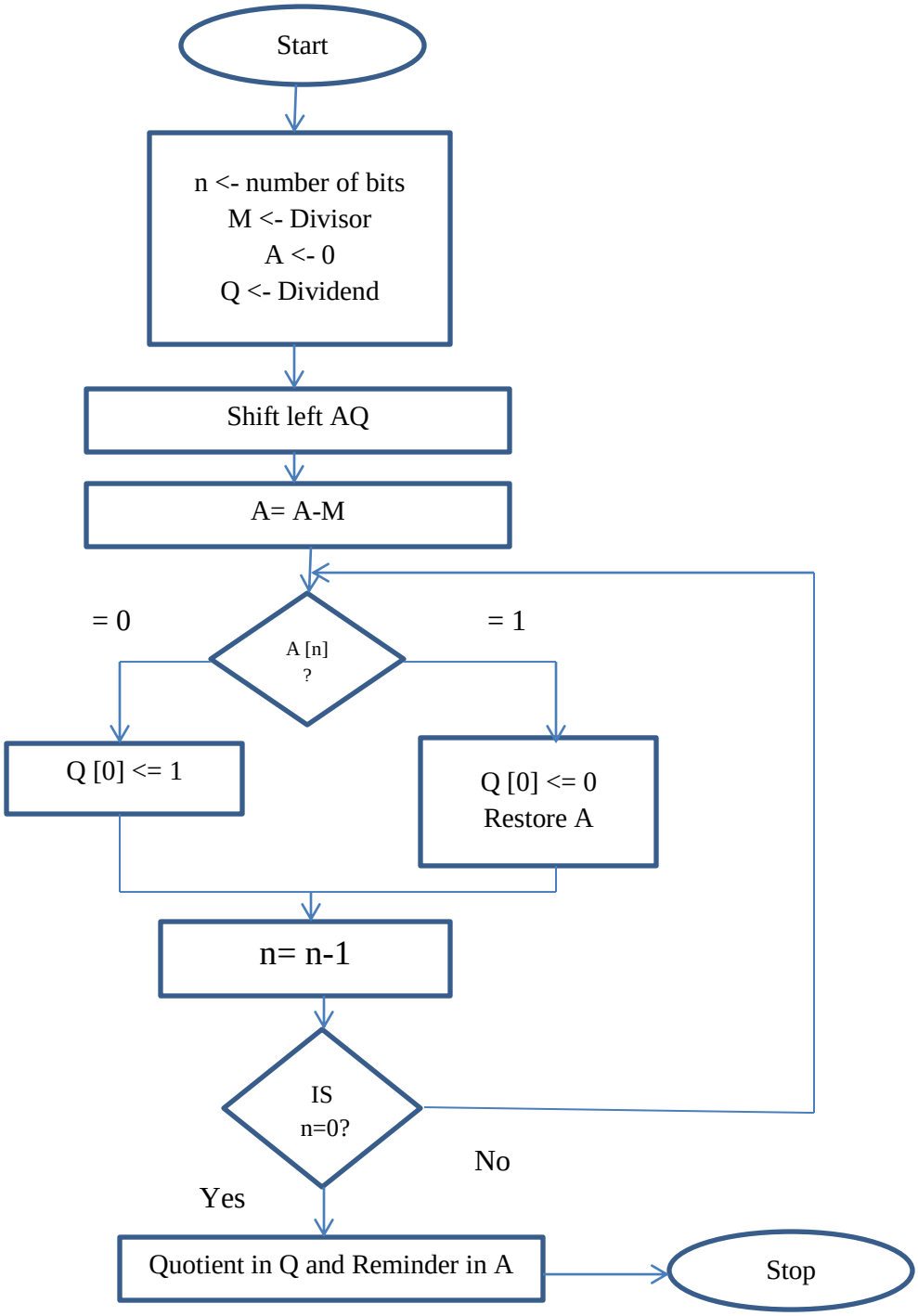
Circuit Diagram:



Restoring Division Algorithm for Unsigned Multiplication

The restoring division algorithm is an approach used to divide one unsigned integer in binary form by another that is in binary form. It performs the division operation by dividing a dividend with a divisor and gives the quotient and remainder.

Flowchart:



Example: 11 (Dividend) / 3 (Divisor) = 3 (Quotient) and 2 (Reminder)

Here, M = 3_{10}

= 00011 (5 bits binary representation)

-M = 11101 (2's Complement)

Tracing Table:

Note: if n= n number of bits, then M and A always n+1 bits.

n	M	A	Q	Action
4	00011	00000	1011	Initialization
		00001	011?	SL AQ
		11110	011?	A=A-M
3		00001	0110	Q[0] <= 0 Restore A
		00010	110?	SL AQ
		11111	110?	A=A-M
2		00010	1100	Q[0] <= 0 Restore A
		00101	100?	SL AQ
		00010	100?	A=A-M
1		00010	1001	Q[0] <= 1
		00101	001?	SL AQ
		00010	001?	A=A-M
0		00010	0011	Q[0] <= 1

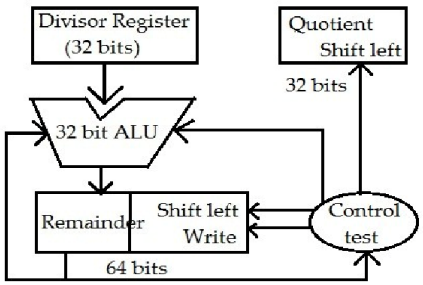
Result: Quotient= 0011_2

= 3_{10}

Reminder= 00010_2

= 2_{10}

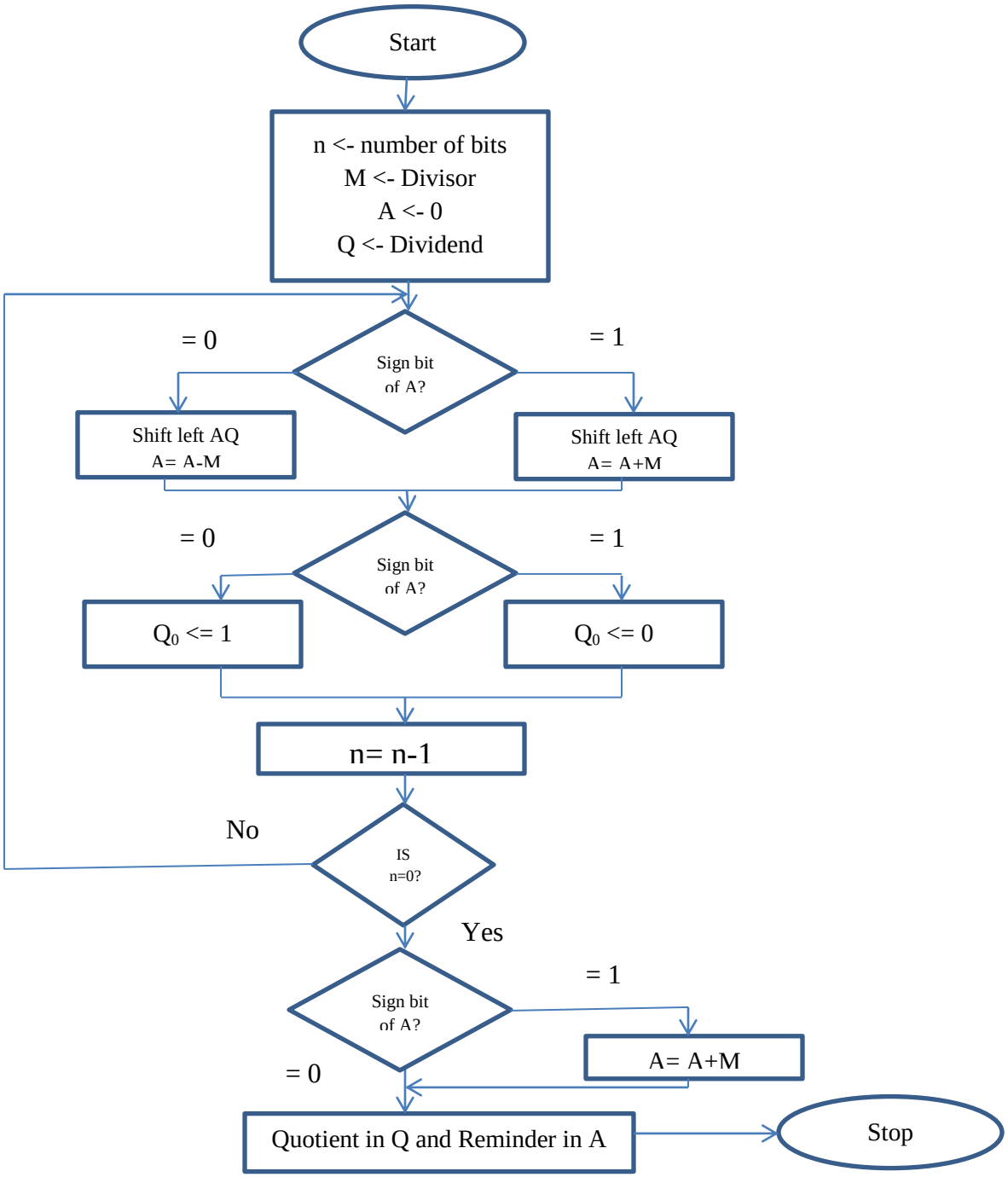
Circuit Diagram:



Non-Restoring Division Algorithm for Unsigned Multiplication

The non-restoring division is a division technique for unsigned binary values that simplifies the procedure by eliminating the restoring phase. The non-restoring division is simpler and more effective than restoring division since it just employs addition and subtraction operations instead of restoring division, which requires extra steps to restore the original result after a failed subtraction.

Flowchart:



Example: 11 (Dividend) / 3 (Divisor) = 3 (Quotient) and 2 (Reminder)

Here, M = 3_{10}

= 00011 (5 bits binary representation)

-M = 11101 (2's Complement)

Tracing Table:

Note: if n= n number of bits, then M and A always n+1 bits.

n	M	A	Q	Action
4	00011	00000	1011	Initialization
		00001	011?	SL AQ
		11110	011?	A=A-M
3		11110	0110	Q[0] <= 0
		11100	110?	SL AQ
		11111	110?	A=A+M
2		11111	1100	Q[0] <= 0
		11111	100?	SL AQ
		00010	100?	A=A+M
1		00010	1001	Q[0] <= 1
		00101	001?	SL AQ
		00010	001?	A=A-M
0		00010	0011	Q[0] <= 1

Result: Quotient= 0011_2

= 3_{10}

Reminder= 00010_2

= 2_{10}

By skipping the restoring phase, the Non-Restoring Division Algorithm provides a more efficient method for dividing unsigned binary values. This simplification makes calculation quicker and more effective, but it also comes with significant drawbacks, especially when working with signed numbers.

Home Work:

Explain some advantages and disadvantages of Restoring and Non-restoring algorithm for division.